

Machine Learning and related Verification Problems

Prof. Sayan Mitra

Adapted from Slides by Prof. Huan Zhang from Spring 2025

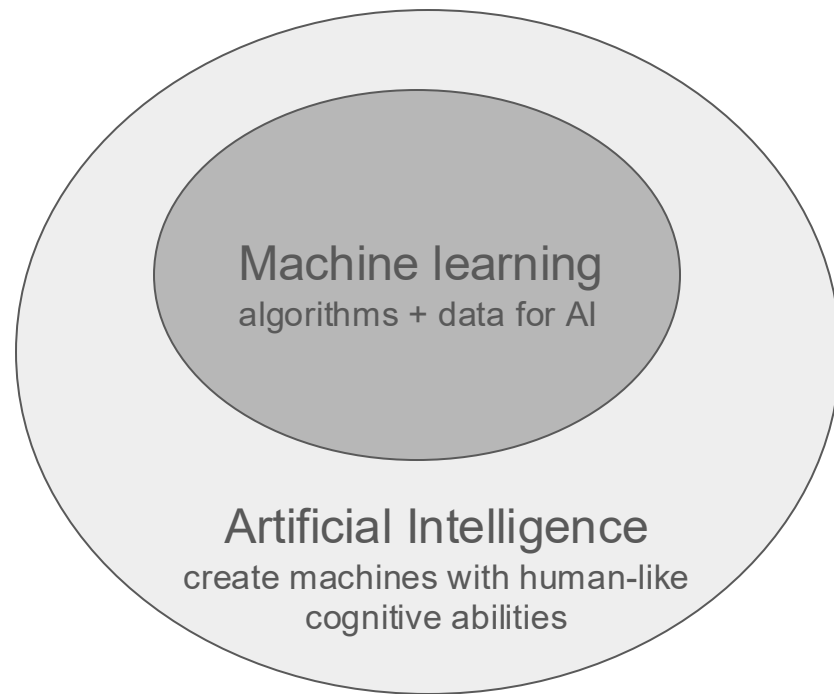
What is machine learning?

“the capacity of computers to **learn** and adapt **without following explicit instructions**, by using **algorithms** and **statistical** models to analyse and infer from patterns in **data**”

-- Oxford English Dictionary

“a **field of study** in **artificial intelligence** concerned with the development and study of **statistical algorithms** that can **learn** from **data** and **generalize** to unseen data, and thus perform tasks **without explicit instructions.**”

--Wikipedia



Example: spam email classification

These are what show up in my Gmail “spam” folder:

» Bid 2	Exclusive Offer: Your NFT Sparks Good Bids ! - February 03, 2024 Read Online Hello, I hope this message finds you...	Feb 3
» Elizabeth	🎁💰 Dont wait any longer - claim your payout now! - 💰 Your journey is leading you to a payout 💰, a reward for all... 📅	Feb 1
» EventPancakeSwap	Join PancakeSwap Airdrop of 135.000\$ Now ! - Join PancakeSwap Exclusive Airdrop Event Hello Valued PancakeSw...	Jan 30
» AceHardware_Winner_	RE: You have won an DEWALT 200 Piece MechanicsToolSet bftHl - Hurry up. The number of prizes to be won is limi...	Jan 29
» Livingston Gym	Thanks for reaching out to us! - Thank you for reaching out to us via our website form at livingstongym.com/contact. ...	Jan 23
» Club1Hotels	🎉 Save an Additional 10% Off Instantly - Plus, up to 20% off E-Gift Cards 🎁 Exclusive Double Offer: Save More on ...	Jan 21

TODO: write a program to classify whether an email is a spam email?

Step 1: collect data

» Bid 2	Exclusive Offer: Your NFT Sparks Good Bids ! - February 03, 2024 Read Online Hello, I hope this message finds you...	Feb 3
» Elizabeth	💰💰 Dont wait any longer - claim your payout now! - 💰 Your journey is leading you to a payout 💰, a reward for all... 📅	Feb 1
» EventPancakeSwap	Join PancakeSwap Airdrop of 135.000\$ Now ! - Join PancakeSwap Exclusive Airdrop Event Hello Valued PancakeSw...	Jan 30
» AceHardware_Winner_	RE: You have won an DEWALT 200 Piece MechanicsToolSet bftHl - Hurry up. The number of prizes to be won is limi...	Jan 29
» Livingston Gym	Thanks for reaching out to us! - Thank you for reaching out to us via our website form at livingstongym.com/contact. ...	Jan 23
» Club1Hotels	😄 Save an Additional 10% Off Instantly - Plus, up to 20% off E-Gift Cards 🎁 Exclusive Double Offer: Save More on ...	Jan 21

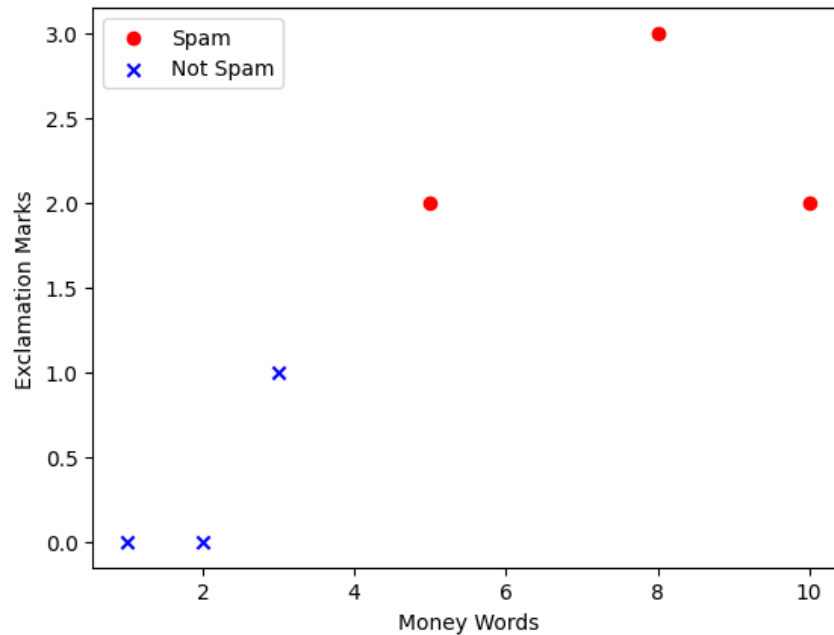
Define some “Features”:

Count of “money words” (“payout”, “\$”, “dollar”, “prizes”, “NFT”, ...)

Count of exclamation marks

Step 1: collect data

Money Words (x_1)	Exclamation Marks (x_2)	Spam (y)
10	2	1
2	0	0
5	2	1
3	1	0
8	3	1
1	0	0



Non-machine-learning approach: write a program explicitly

Define some “Features”:

Count of “money words” (“payout”, “\$”, “dollar”, “prizes”, “NFT”, ...)

Count of exclamation marks

```
def is_spam(count_money_word, count_exclamation_mark):  
    if a * count_money_word + b * count_exclamation_mark >= threshold:  
        return True  
    else:  
        return False
```

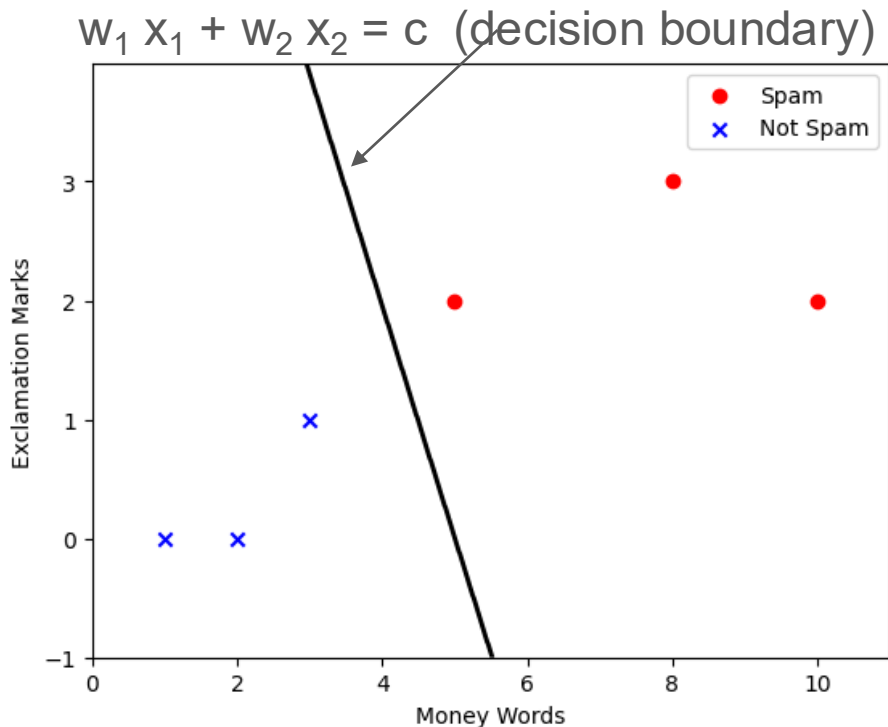
A human programmer chooses `a`, `b`, `threshold`

Step 2: learning from data (model training)

Instead of choosing these parameters, we learn these from data

Algorithms to find this classifier (not discussed in this class):

- Logistic regression
- Support vector machines



Step 3: prediction

Given **model weights** $w \in \mathbb{R}^N$

Given **features** of an input $x \in \mathbb{R}^N$

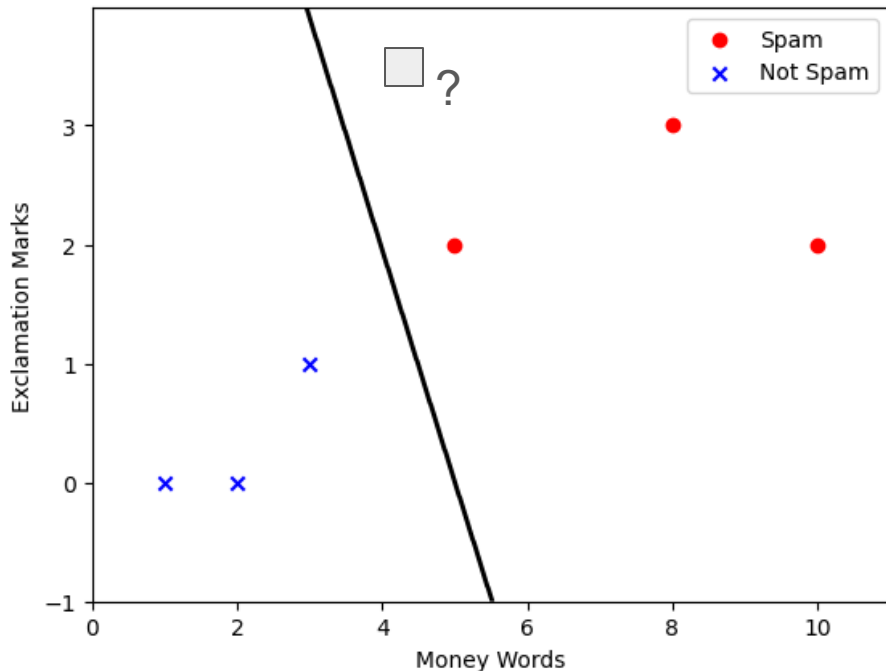
If $w^T x > c$: predict positive class (e.g., spam)

If $w^T x < c$: negative class (e.g., not spam)

Note that by simple transformations on w and x , we just need to check $w'^T x' > 0$ or $w'^T x' < 0$:

$$w' = [w_1, \dots, w_N, -c]$$

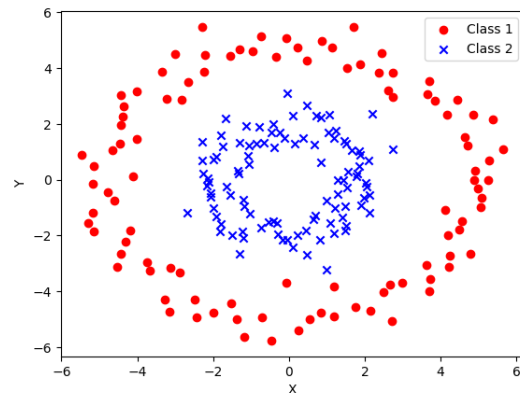
$$x' = [x_1, \dots, x_N, 1]$$



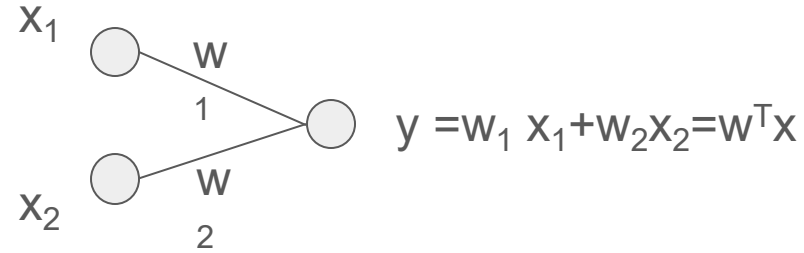
When linear function does not work well

To solve most practical classification problems, non-linear classifiers are needed. Many different approaches:

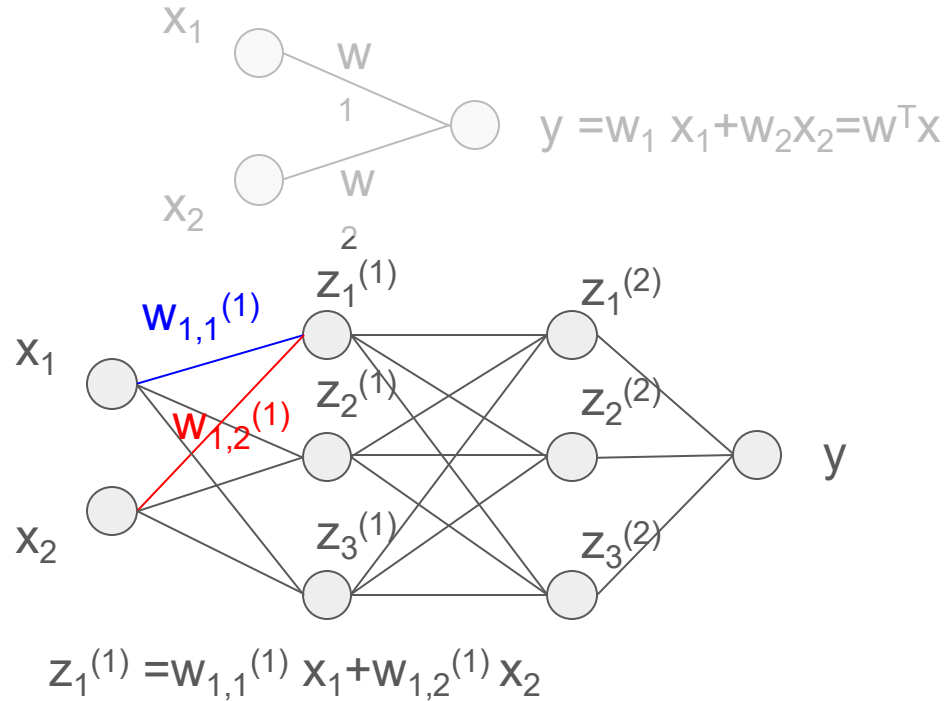
- Kernel method
- **Neural networks**
- Tree ensembles
- ...



Neural Networks: let's just stack linear functions multiple times?



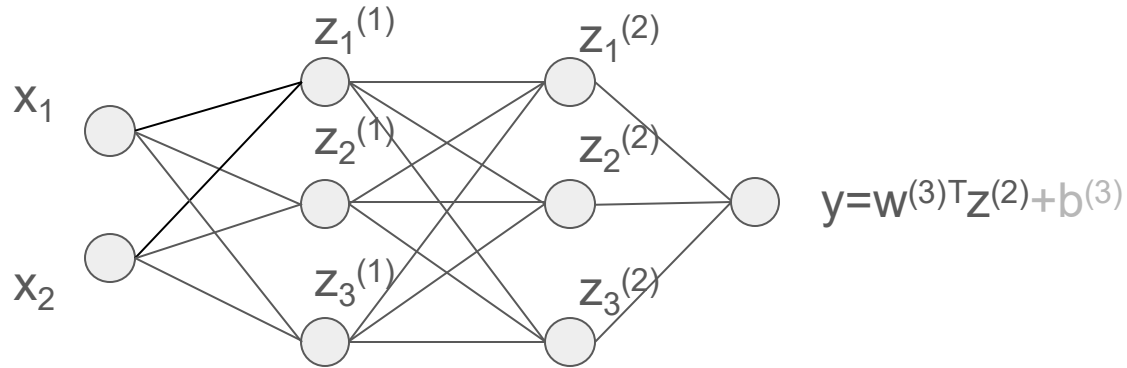
Neural Networks: let's just stack linear functions multiple times?



In general we write in matrix form: $z^{(1)} = W^{(1)}x$, $W^{(1)}$ is a 3×2 matrix above

Neural Networks: let's just stack linear functions multiple times?

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad z^{(2)} = W^{(2)}z^{(1)} + b^{(2)} \quad \text{A bias term can be added}$$

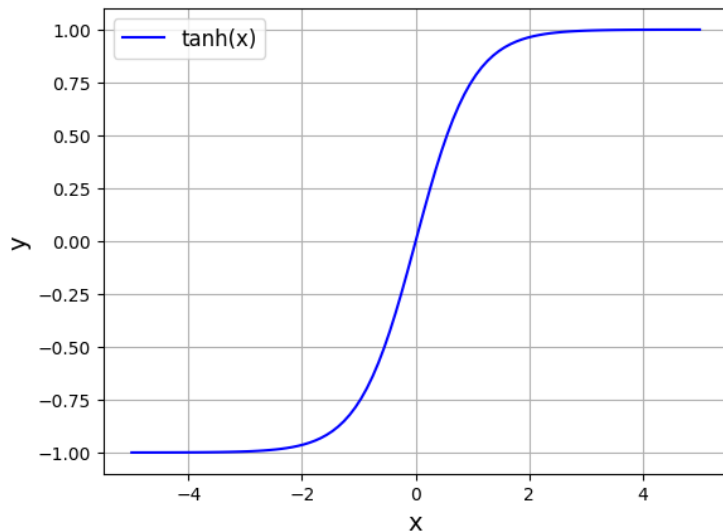
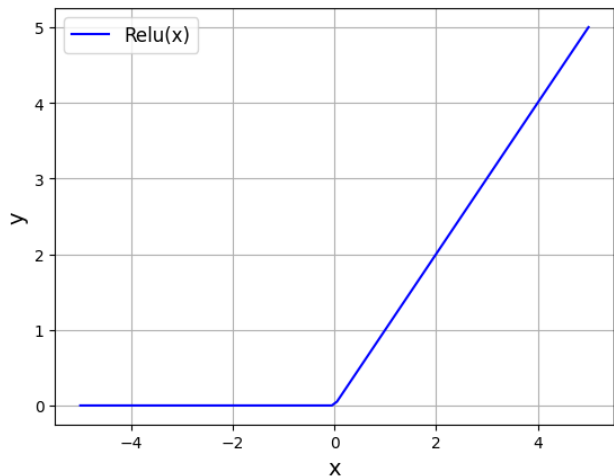


$$y = w^{(3)T}z^{(2)} = w^{(3)T}W^{(2)}z^{(1)} = \mathbf{w^{(3)T}W^{(2)}W^{(1)}}x \quad \text{still a linear function of } x!$$

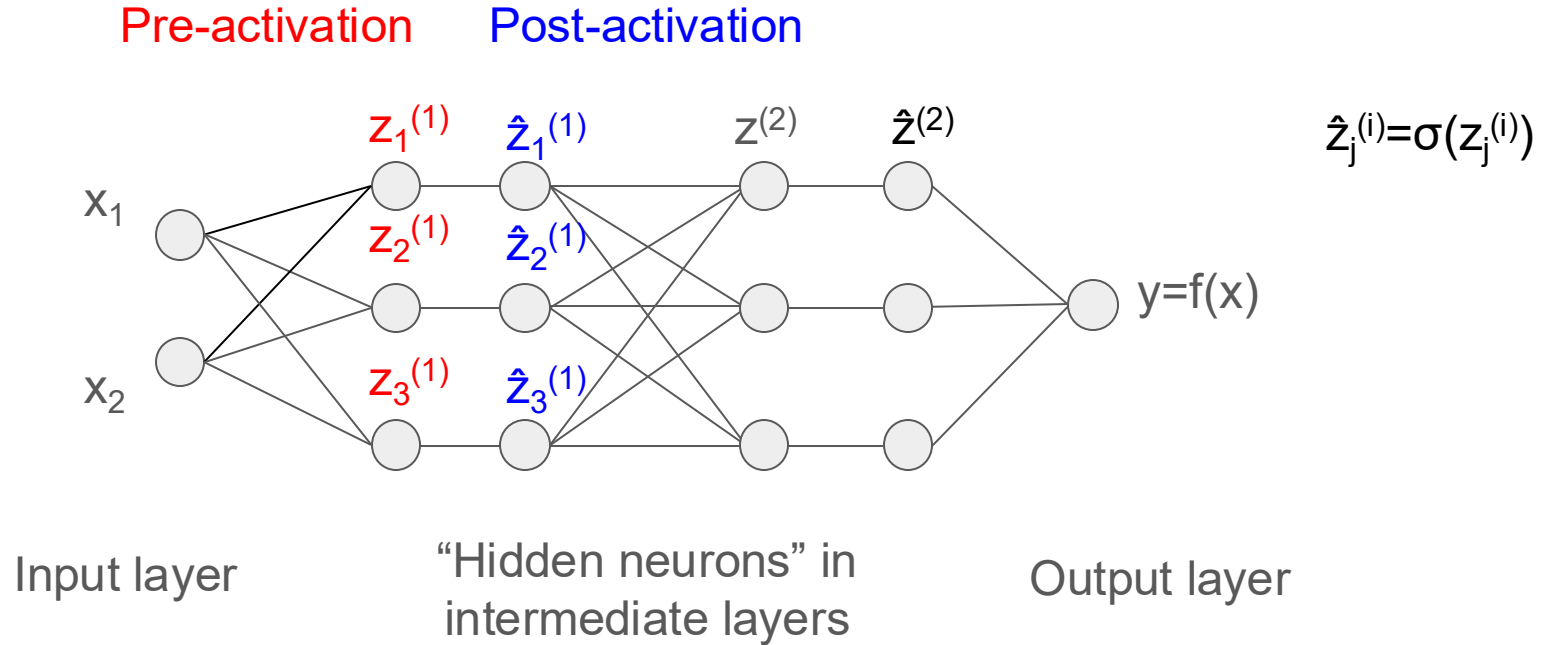
Must introduce nonlinear functions (“activation” functions)

ReLU: rectified linear unit

$$\text{ReLU}(x) := \max(0, x)$$

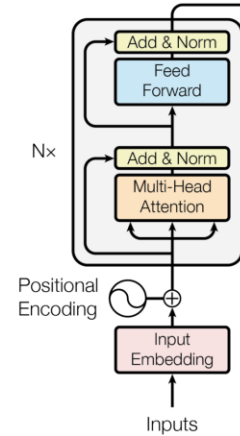
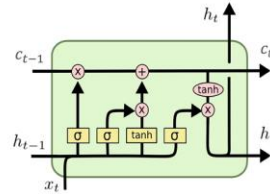
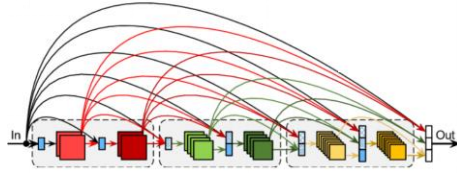


Neural Networks: linear + non-linear layers (multi-layer perceptron)

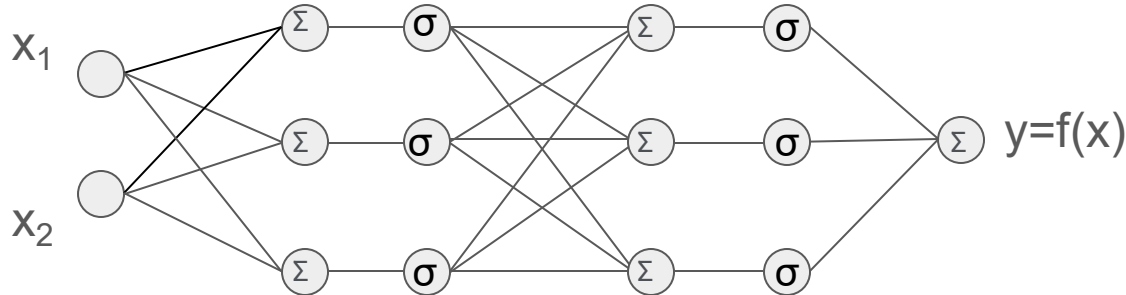


Neural networks are “**Universal approximators**”

Many other neural network architectures available



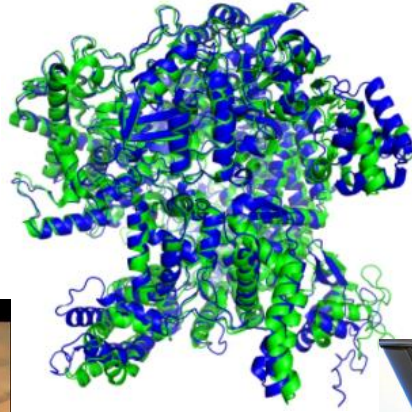
In general, neural networks can be presented as a “computation graph”



Many other neural network architectures available



AlphaGo (2020)



AlphaFold (2021)



“A robot manipulating an aircraft”

Stable Diffusion (2022)

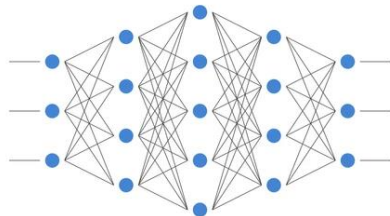


ChatGPT, LLMs
(2023+)

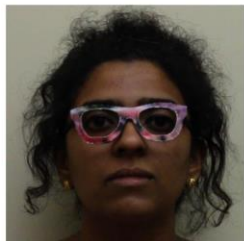
Neural networks have fragile decision boundaries (adversarial inputs)



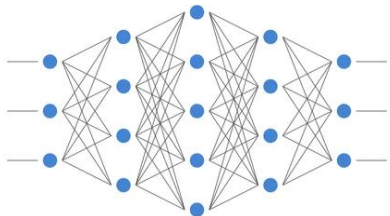
Eykholt et al. 2018



“Speed limit 45”



Sharif et al. 2018



“Brad Pitt”

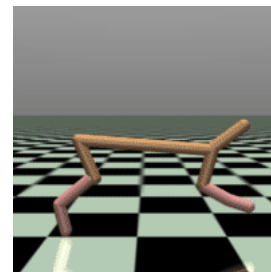
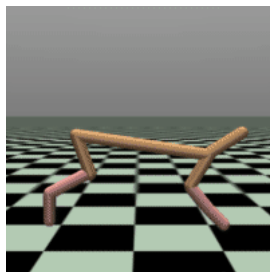
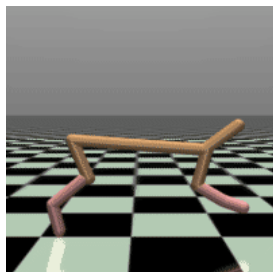


“Adversarial examples”

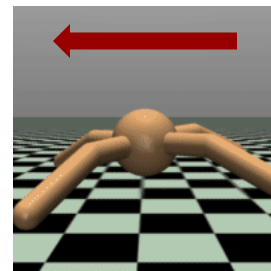
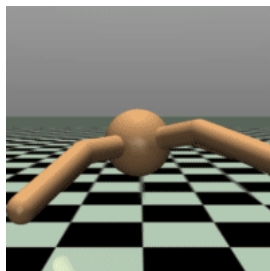
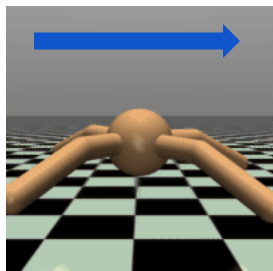
Neural networks are not safe enough for mission-critical tasks

Neural network controlled robots (simulated) + adversarial sensor noise

HalfCheetah



Ant



No attack

MAD attack

Optimal attack

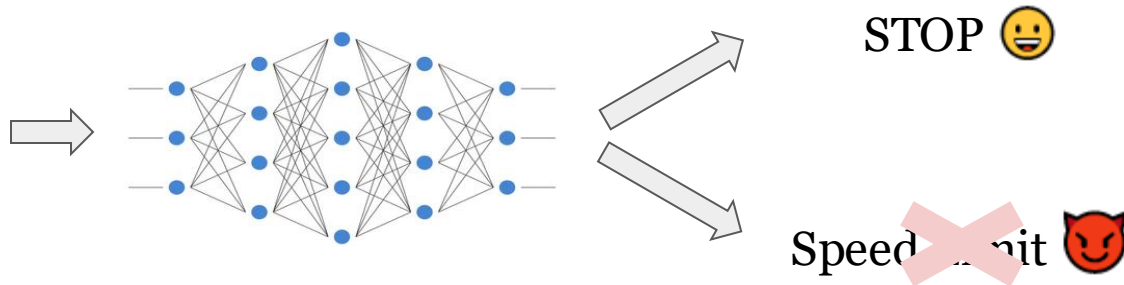
[Z*C*XLLBH NeurIPS 2020]

[Z*CBH ICLR 2021]

Formal verification of neural networks: robustness verification



Eykholt et al. 2018

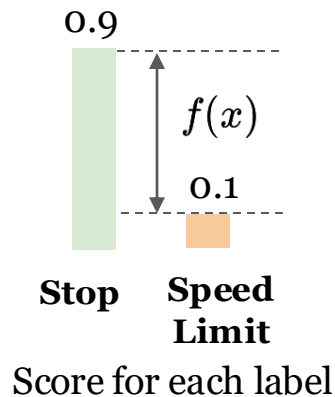
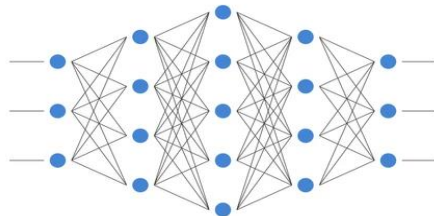


Goal: **prove** adversarial examples do *not* exist!

Formal verification of neural networks: robustness verification



Attacker may put *anything* here



$f(x) > 0 \Rightarrow$ No adversarial examples

Formal verification of neural networks: robustness verification

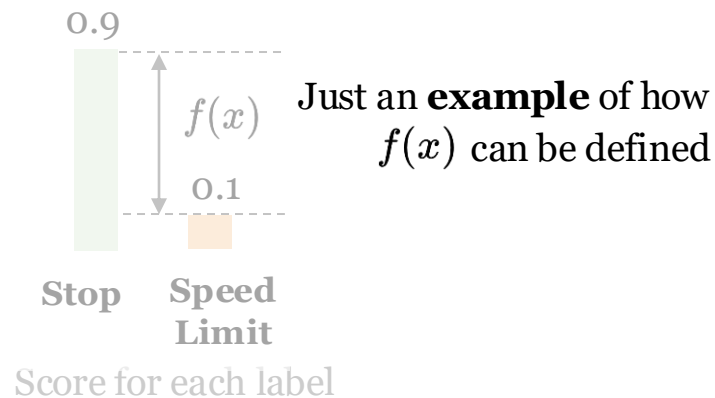
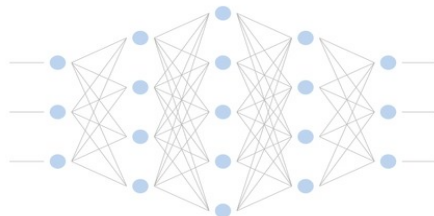


Attacker may put *anything* here

Just an **example** of verification problem

Prove: $\forall x \in \mathcal{S}, f(x) > 0$

For multi-class cases, we can define multiple $f_i(x)$, one for each class



$\mathcal{S}$ = all possible pixel perturbations



$x_1 \in \mathcal{S}$



$x_2 \in \mathcal{S}$



$x_3 \in \mathcal{S}$

...

Verification example: ACAS Xu system

3MB DNN represents a large (2GB) lookup table for collision avoidance of unmanned aircraft

Input: $x \in \mathbb{R}^5$, $x = (d, \theta, \psi, v_{\text{own}}, v_{\text{in}})$

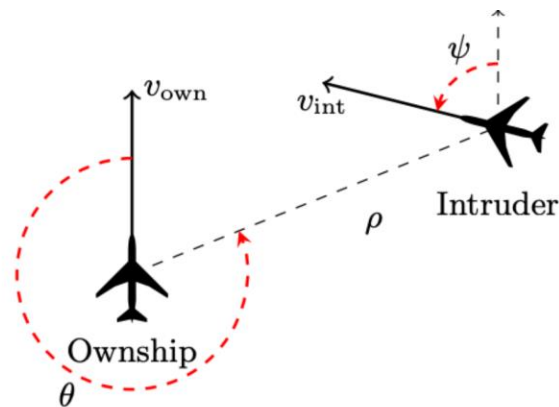
d : Distance; θ : relative angle; ψ : relative heading; v_{own} , v_{in} : speeds

Output $y \in \mathbb{R}^5$: Clear of Conflict (COC), or advisory weak/strong left/right. Five scores for these actions:

y_0 : COC, y_1 : weak left at 1.5 deg/s

y_2 : strong left at 3.0 deg/s

y_3 : weak right y_4 : strong right



“Neural Network Verification Methods for Closed-Loop ACAS Xu Properties”, Bak et. al.

Verification example: ACAS Xu system

3MB DNN represents a large (2GB) lookup table for collision avoidance of unmanned aircraft

Input: $x \in \mathbb{R}^5$, $x = (d, \theta, \psi, v_{\text{own}}, v_{\text{in}})$

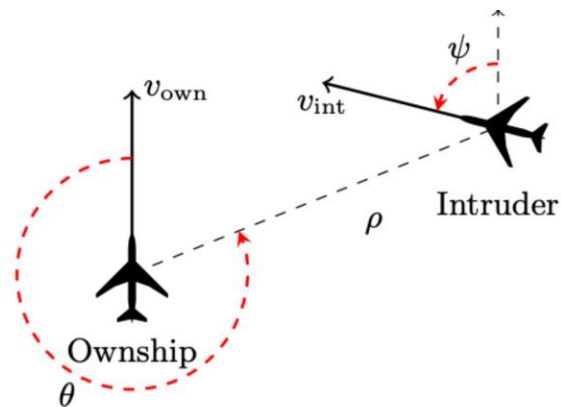
d : Distance; θ : relative angle; ψ : relative heading; v_{own} , v_{in} : speeds

Output $y \in \mathbb{R}^5$: Clear of Conflict (COC), or advisory weak/strong left/right.

Requirement: E.g. If the intruder is far then the score for COC should be above some threshold

$\forall x \in \mathbb{R}^5, d \geq 55947, v_{\text{own}} \geq 1145, v_{\text{in}} \leq 60$

Prove: $y_0 > 1500$



“Neural Network Verification Methods for Closed-Loop ACAS Xu Properties”, Bak et. al.

Verification example: Neural Network Controlled Systems

Controller input x : angle θ , angular velocity $\dot{\theta}$

Controller output: torque u applied to pendulum

Requirement: the controller can stabilize the pendulum ($\theta = \dot{\theta} = 0$) eventually

Need to use Lyapunov theory to verify that the “energy” of the system keeps decreasing over time.

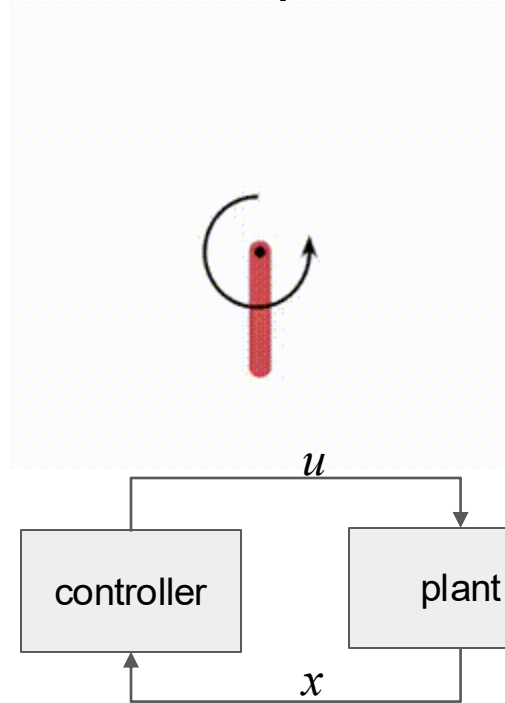
$$V(x_{t+1}) - V(x_t) \leq 0, \text{ for all } x_t \in \mathcal{S}$$

$$x_{t+1} = f(x_t, \pi(x_t))$$

Physical system
dynamics

Controller NN

“Region of attraction”



“Lyapunov-stable Neural Control for State and Output Feedback: A Novel Formulation”, Yang et. al.

“Neural Lyapunov control”, Chang et al.

Verification of neural networks

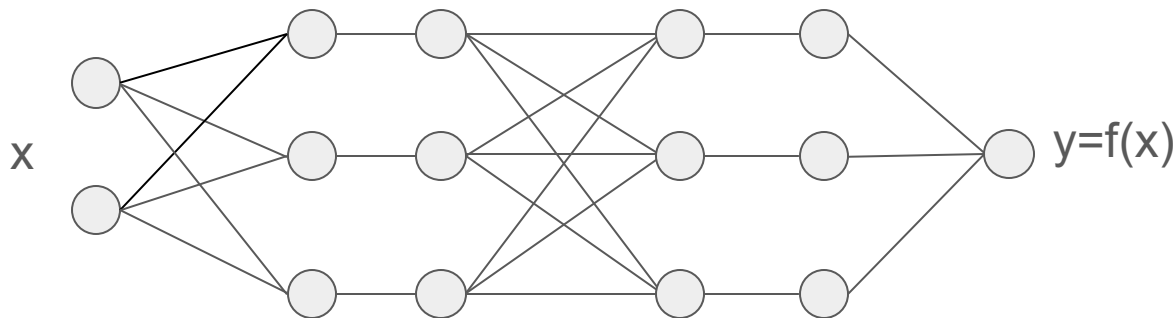
For all desired input x (image, text, sensor readings, etc), $f(x)$ meets some conditions

Satisfiability problem: does there *exist* x , such that $f(x)$ does not meet these conditions?



$$x \in S \wedge y \leq 0 \wedge y = f(x)$$

Can also be multiple conditions, like in some ACAS Xu requirements and robustness verification of multi-class classification



Verification example: ACAS Xu system (from VNN-COMP)

Input: $x \in \mathbb{R}^5$, $x = (d, \theta, \psi, v_{\text{own}}, v_{\text{in}})$

d : Distance; θ : relative angle; ψ : relative heading; $v_{\text{own}}, v_{\text{in}}$: speeds

Output $y \in \mathbb{R}^5$: y_0 : COC, y_1 : weak left, y_2 : strong left, y_3 : weak right, y_4 : strong right

```
; Unscaled Input 0: (55947.691, 60760)
(assert (<= X_0 0.679857769))
(assert (>= X_0 0.6))

; Unscaled Input 1: (-3.141592653589793, 3.141592653589793)
(assert (<= X_1 0.5))
(assert (>= X_1 -0.5))

; Unscaled Input 2: (-3.141592653589793, 3.141592653589793)
(assert (<= X_2 0.5))
(assert (>= X_2 -0.5))

; Unscaled Input 3: (1145, 1200)
(assert (<= X_3 0.5))
(assert (>= X_3 0.45))

; Unscaled Input 4: (0, 60)
(assert (<= X_4 -0.45))
(assert (>= X_4 -0.5))

; Unsafe if COC is maximal
(assert (<= Y_1 Y_0))
(assert (<= Y_2 Y_0))
(assert (<= Y_3 Y_0))
(assert (<= Y_4 Y_0))
```

Requirements written in VNNLIB format

multiple conditions on y

$$(y_1 - y_0 \leq 0) \wedge (y_2 - y_0 \leq 0) \wedge (y_3 - y_0 \leq 0) \wedge (y_4 - y_0 \leq 0)$$

Verification of neural networks

Does there *exist* x , such that:

$$x \in S \wedge y \leq 0 \wedge y = f(x)$$

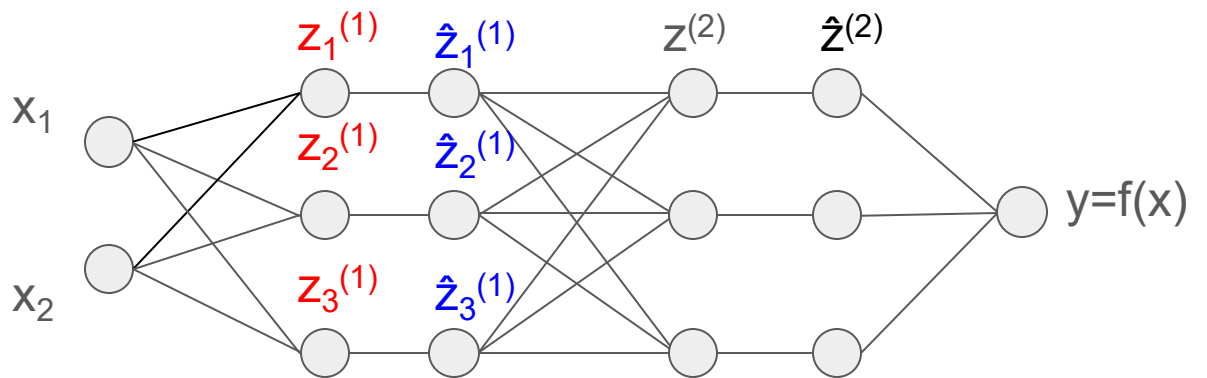
$x \in S$ condition is easy to handle for box constraints:

$$x_i \leq u_i \wedge x_i \geq l_i$$

How to handle $y = f(x)$?

Verification of neural networks

How to handle the constraint $y = f(x)$?

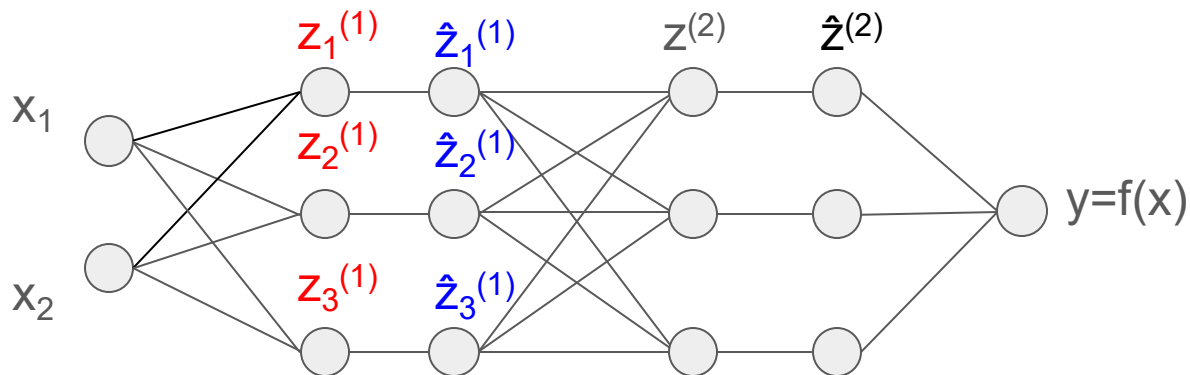


$$\hat{z}_j^{(i)} = \sigma(z_j^{(i)})$$

Linear layers: $z^{(1)} = W^{(1)} x$ $z^{(2)} = W^{(2)} \hat{z}^{(1)}$ $y = w^{(3)T} \hat{z}^{(2)}$

Verification of neural networks

How to handle the constraint $y = f(x)$?

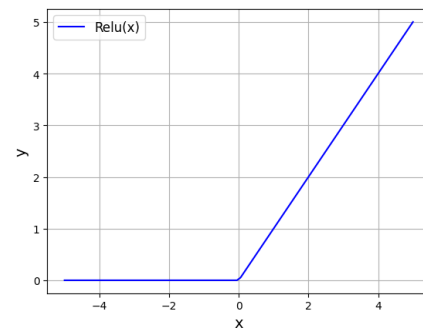
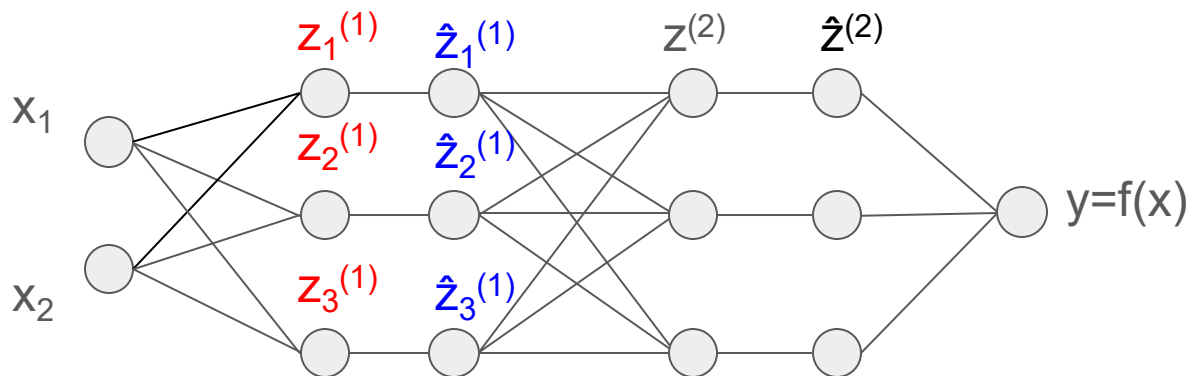


Linear layers: $z^{(1)} = W^{(1)} x$ $z^{(2)} = W^{(2)} \hat{z}^{(1)}$ $y = w^{(3)T} \hat{z}^{(2)}$

Directly copy all the linear equality constraints to the SMT formulation.

Verification of neural networks

How to handle the constraint $y = f(x)$?



$$\hat{z}_j^{(i)} = \text{ReLU}(z_j^{(i)}) \Rightarrow (z_j^{(i)} \geq 0 \wedge \hat{z}_j^{(i)} = z_j^{(i)}) \vee (z_j^{(i)} < 0 \wedge \hat{z}_j^{(i)} = 0)$$

Verification of neural networks

Satisfiability problem: $\exists x \in S \wedge y \leq 0 \wedge y = f(x)$

$x_i \leq u_i \wedge x_i \geq l_i$ for each dimension of x

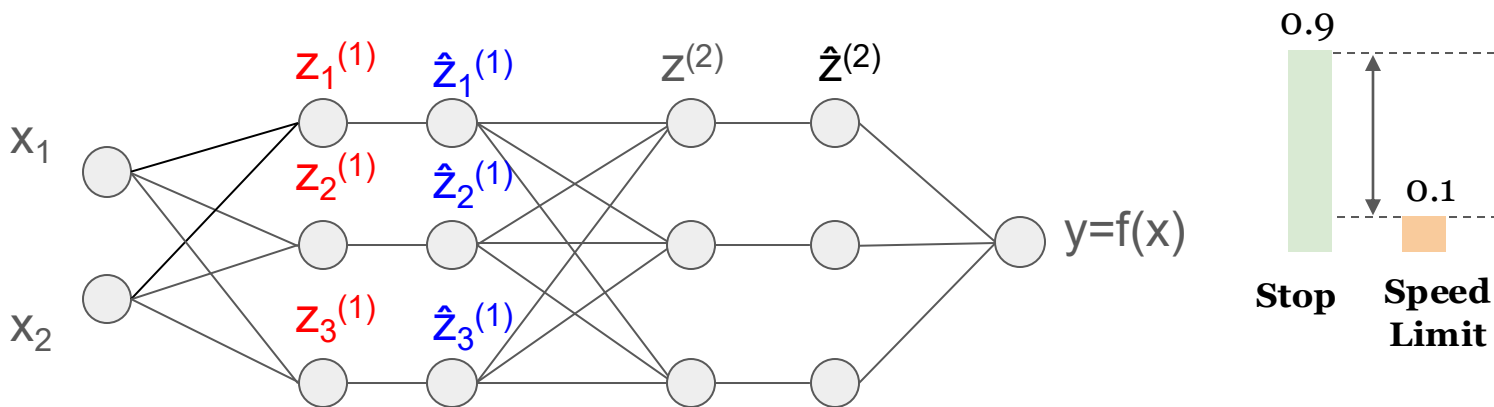
$((z_j^{(i)} \geq 0 \wedge \hat{z}_j^{(i)} = z_j^{(i)}) \vee (z_j^{(i)} < 0 \wedge \hat{z}_j^{(i)} = 0))$ for each ReLU neuron

$z_1 = W^{(1)} x \wedge z^{(2)} = W^{(2)} \hat{z}^{(1)} \wedge y = w^{(3)T} \hat{z}^{(2)} \wedge y \leq 0$

Add all clauses to the formula and solve using DPLL(T) with **Linear Real Arithmetic**.

In general this is very slow!

Summary: neural network verification as a satisfiability problem



Does there exist x , s.t. $x \in S \wedge y \leq 0 \wedge y = f(x)$

Input domain under
consideration

Negation of the
desired property

Defines the neural
network

Summary

- Please checkout **verification of neural networks competitions** (VNN-COMP) for more examples of verification problems
 - <https://sites.google.com/view/vnn2023>
 - <https://sites.google.com/view/vnn2022>
 - <https://sites.google.com/view/vnn2021>
- Next lecture: integer programming and linear programming formulations for neural network verification
- **Reading:**
 - <https://arxiv.org/pdf/1711.07356.pdf>
 - <https://arxiv.org/pdf/1711.00851.pdf>

References

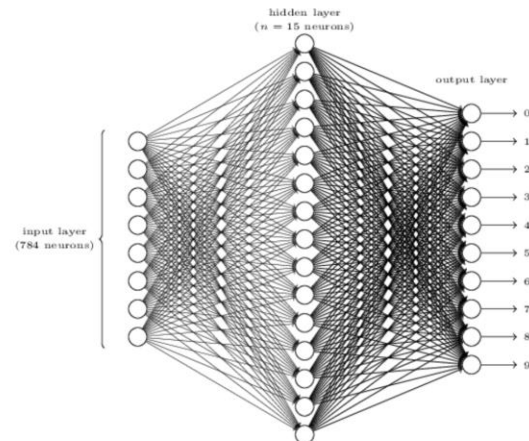
- Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks by Guy Katz, Clark Barrett, David Dill, Kyle Julian, Mykel Kochenderfer, CAV 2017
- Book: Introduction to Neural Network Verification by Aws Albarghouthi, 2021
- Further exploration: Neural Network Verification competition:
<https://sites.google.com/view/vnn20/>
- **Neural Networks and Deep Learning**

Neural networks as graphs

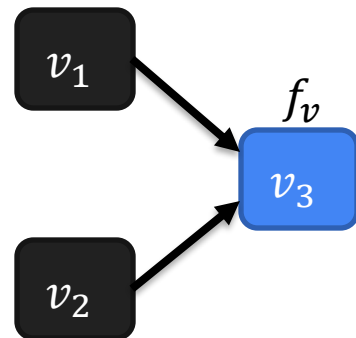
- Consider a *network* $G = (V, E)$
- Let $V^{in}, V^o \subseteq V$ be nonempty sets of input, output vertices
- Each $v \in V \setminus V^{in}$ is associated with a function $f_v: \mathbb{R}^{ind(v)} \rightarrow \mathbb{R}$ where $ind(v)$ is the indegree of v
- Typically these functions are differentiable (almost everywhere)

Assume w.l.o.g

- All nodes are reachable from some V^{in}
- Every node reach some V^o
- There is a total ordering on V and another on E

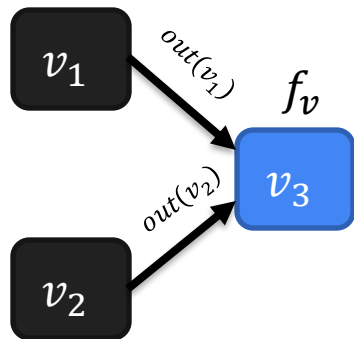


<http://neuralnetworksanddeeplearning.com/chap1.html>



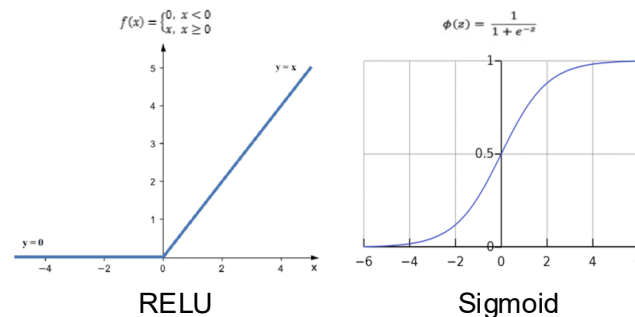
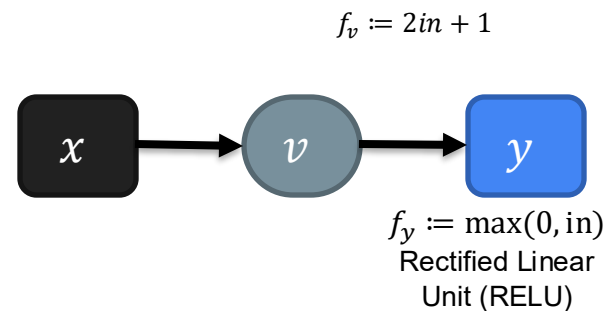
Computation performed by a Network

- $|V^{in}| = n, |V^o| = m$
- The network $G = (V, E)$ defines a function $\mathbb{R}^n \rightarrow \mathbb{R}^m$ defined as follows
- For any non input node $v \in V$ the output from v is recursively defined as follows: let $(v_1, v), (v_2, v), \dots, (v_{ind(v)}, v)$ be the ordered set of all input edges to v
$$out(v) := f_v(out(v_1), \dots, out(v_{ind(v)}))$$
 - Base case: For input vector $x \in \mathbb{R}^n$, for any input node $v \in V^{in}$, output $out(v) := x_i$



Example functions

$$\begin{aligned} out(y) &= f_y(f_v(out(x))) \\ &= f_y(f_v(x)) \\ &= f_y(2x + 1) \\ &= \max(0, 2x + 1) \end{aligned}$$



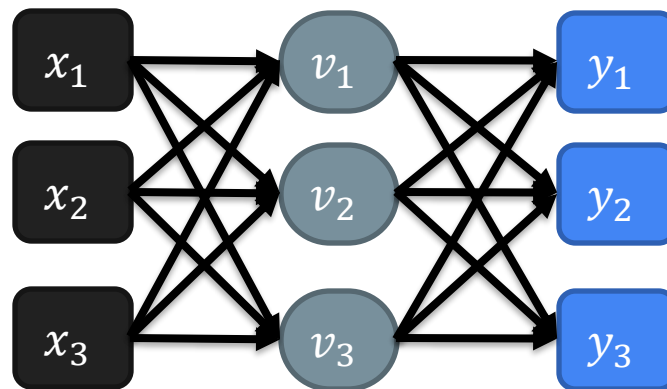
$$f_y := \sigma(in) = \frac{1}{1 + \exp(-in)}$$

Example functions

$$\begin{aligned} out(y_1) &= f_y \left(\sum_{i=1}^3 out(v_i) w_i \right) \\ &= f_y \left(\sum_{i=1}^3 \left(\sum_{j=1}^3 x_j u_j \right) w_i \right) \end{aligned}$$

Multi-layered Perceptron

Hidden layer



$$f_{y_i} := \frac{\text{Softmax}}{\sum_{i=1}^3 \exp(v_i)} \exp(v_i)$$

Typical requirements studied for neural networks

For any input x (image, text, program, controller) $f(x)$ meets <some condition>

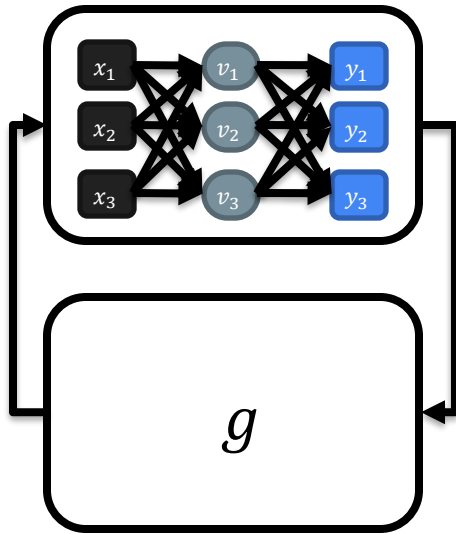
To check the validity of this statement, we can check the satisfiability of the negation, i.e., does there exist some input x such that $f(x)$ does not meet <some condition>

For any inputs x, y (image, text, program, controller) $f(x)$ and $f(y)$ meets <some condition>

Pros. Same form as pre-postconditions of automata transitions

Cons. with this framing

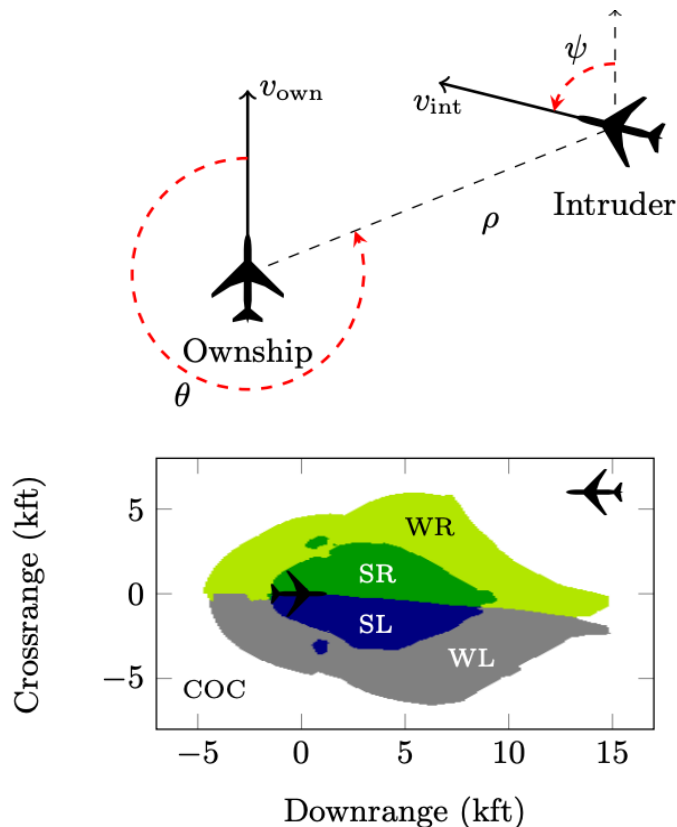
- <some condition> usually requires domain knowledge
- can only be specified *locally*
- Does not cover requirements that use the NN in closed-loop system



Application: ACAS Xu system

- 3MB DNN represents a large (2GB) lookup table for collision avoidance of unmanned aircraft
- Input: $x \in \mathbb{R}^7$ ρ : Distance; θ : relative angle; ψ : relative heading; v_{own} , v_{in} : Speeds, τ : Time until loss of vertical separation; a_{prev} : Previous advisory.
- Output: Clear of Conflict (COC) or advisor weak/strong left/right.
- Network: 45 networks produced by discretizing τ and a_{prev} , each with 5 inputs and 5 outputs
- Requirement: E.g. If the intruder is far then the score for COC should be above some threshold

$$\forall x \in \mathbb{R}^7 \quad x.d > 55947, x.v_{own} > 1145, x.v_{in} \leq 60 \Rightarrow f(x) \geq 1500$$



General formulation of correctness

$$(precondition \wedge neural\ network) \Rightarrow postcondition$$

$$\left(\bigwedge_{i=1}^n |x_i - c_i| \leq 0.1 \right) \wedge F_G \bigwedge \left(\bigwedge_{i=1}^n x_i = v_i^o \right) \wedge \left(\bigwedge_{i=k}^{k+m} r_i = v_i^o \right) \wedge \left(\bigwedge_{i=1}^m r_1 > r_{1+i} \right)$$

Precondition P

$$r_1 == f_{G1}(x_1)$$

$$r_2 == f_{G2}(x_2)$$

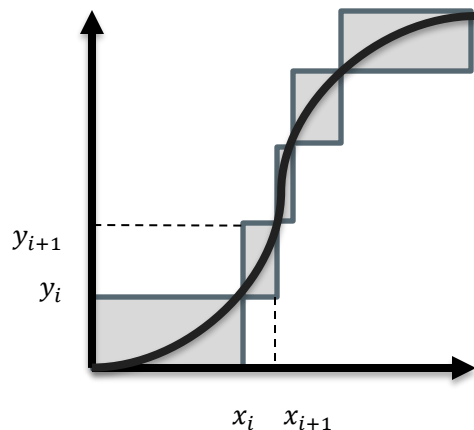
...

$$r_l == f_{Gl}(x_l)$$

Postcondition Q

Sigmoids and ReLUs and exponential blow-up

- Sigmoids can be approximated by piece-wise relations
- Others have considered MILP encodings
- Without disjunctions the problem is an LP
 - Polynomial Time solvable, Simplex
- If for all the inputs, the ReLUs are either active (x) or inactive (0)
 - We can replace each ReLU with either $x_i = f(x)$ or $x_i = 0$ based on lightweight analysis of which is which
- Otherwise, DPLL has to handle the disjunctions and in the worst case will have to consider every possible case (active or inactive) for every possible ReLU leading to exponential number of calls to Simplex
- Reluplex [Katz et al. 2017] addresses this problem



Reluplex Decision Procedure for ReLU NNs

Input: F in Reluplex form

Output: $\exists x \in \mathbb{R}^m$ such that $x \models F$?

- Delay case splitting on ReLUs
- In the worst case it is still exponential, but has been shown to be empirically better than DPLL^{LRA}

Reluplex Decision Procedure for ReLU NNs

Input: F in Reluplex form

Output: $\exists x \in \mathbb{R}^m$ such that $x \models F$?

Reluplex form

- Equations (same as Simplex)
- Bounds (same as Simplex)
- Relu: $x_i = \text{relu}(x_j)$

Given conjunction of inequalities and ReLU constraints we can convert them to Reluplex form and add $x_i \geq 0$.

Reluplex

Input: A formula F in Reluplex form

Output: $x \models F$ or UNSAT

$x := \langle x_i \mapsto 0 \rangle$; $F' :=$ non ReLU part of F

while true **do**

$r := \text{Simplex}(F', x)$

if r is unsat **then return** UNSAT

else if $r \models F$ **then return** r

// Handle violated ReLU constraints

 Let $x_i = \text{relu}(x_j) \in F$ such that $x[x_i \neq \text{relu}(x[x_j])$

// Case split

if $u_j > 0, l_j < 0$ and $x_i = \text{relu}(x_j)$ considered $> \tau$ times

Reluplex

Input: A formula F in Reluplex form

Output: $x \models F$ or UNSAT

$x := \langle x_i \mapsto 0 \rangle$; $F' :=$ non ReLU part of F

while true **do**

$r := \text{Simplex}(F', x)$

if r is unsat **then return** UNSAT

else if $r \models F$ **then return** r

// Handle violated ReLU constraints

 Let $x_i = \text{relu}(x_j) \in F$ such that $x[x_i \neq \text{relu}(x[x_j])$

if x_i is basic **then** pivot x_i with non-basic variable x_k where $k \neq j$ and $c_{ik} \neq 0$

if x_j is basic **then** pivot x_j with non-basic variable x_k where $k \neq i$ and $c_{jk} \neq 0$

$x[x_i := \text{relu}(x[x_j])$ **OR** $x[x_j := x[x_i]$

// Case split

if $u_j > 0, l_j < 0$ and $x_i = \text{relu}(x_j)$ considered $> \tau$ times

$r_1 = \text{Reluplex}(F \wedge x_j \geq 0 \wedge x_j = x_j)$

$r_2 = \text{Reluplex}(F \wedge x_j \leq 0 \wedge x_i = 0)$

if $r_1 = r_2 = \text{unsat}$ **then return** UNSAT

else if $r_1 = \text{unsat}$ **then return** r_1 **else return** r_2

Termination

- The last part of the algorithm ensures termination
- Otherwise, the algorithm could loop forever between fixing relu constraints and Simplex
- $F \equiv x_i = \text{relu}(x_j)$ is split into two cases
 - $F_1 \equiv x_j \geq 0 \wedge x_i = x_j$
 - $F_2 \equiv x_j \leq 0 \wedge x_i = 0$
 - $F' \equiv (F \wedge F_1) \vee (F \wedge F_2)$

Performance of Reluplex on ACAS

	φ_1	φ_2	φ_3	φ_4	φ_5	φ_6	φ_7	φ_8
CVC4	-	-	-	-	-	-	-	-
Z3	-	-	-	-	-	-	-	-
Yices	1	37	-	-	-	-	-	-
MathSat	2040	9780	-	-	-	-	-	-
Gurobi	1	1	1	-	-	-	-	-
Reluplex	8	2	7	7	93	4	7	9

